

Reviewer Pack — Minimal Rank of Tensor Product Algebras and Communication Co...

Entry a62ecea5349a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 13:59:38 UTC

Minimal Rank of Tensor Product Algebras and Communication Complexity of XOR-AND Networks

Entry ID: a62ecea5349a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 13:59:38 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics **Field B** (complexity object): Communication Complexity: XOR-AND Networks

Statement:

[‘For all instances with n variables, the communication complexity of XOR-AND networks is upper bounded by $O(n^2 \log(n) / \log(\log(n)))$ if and only if the minimal rank of the tensor product algebra associated with the instance is less than or equal to some constant c .’, ‘There exists a constant c such that for any XOR-AND network, its communication complexity is at most $O(n^2 \log(n) / \log(\log(n)))$ if the minimal rank of the tensor product algebra constructed from the network’s incidence matrix is less than or equal to c .’]

Rationale (proposer’s reasoning):

[‘The study of tensor product algebras in algebraic combinatorics provides a structured way to represent complex systems, which might capture the inherent structure of XOR-AND networks. If the communication complexity of XOR-AND networks can be related to the minimal rank of their associated tensor product algebras, it could lead to new algorithms for solving communication problems.’, ‘This bridge may expose novel structures in both algebraic combinatorics and communication complexity, potentially leading to improved understanding and algorithm design.’]

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): aa416c3004d2add5

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean minimal rank across all seeds is less than or equal to a constant c , and the average communication complexity for XOR-AND networks with n variables does not exceed $O(n^2 \log(n) / \log(\log(n)))$. The conjecture is falsified if either condition fails.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tensor product algebra AND minimal rank AND communication complexity
XOR-AND networks - incidence matrix AND XOR-AND networks AND communication complexity
 $O(n^2 \log(n) / \log(\log(n)))$ - algebraic combinatorics AND tensor algebra AND communication complexity bound

Top relevant hits considered: - [<http://arxiv.org/abs/1705.09379v4>] Tensor rank is not multiplicative under the tensor product - [<http://arxiv.org/abs/1712.02454v2>] On the structures of hive algebras and tensor product algebras for general linear groups of low rank - [<http://arxiv.org/abs/1102.2932v2>] Applications of Monotone Rank to Complexity Theory - [<http://arxiv.org/abs/2410.13515v2>] Observation of a rare beta decay of the charmed baryon with a Graph Neural Network - [<http://arxiv.org/abs/1511.06033v3>] EigenRec: Generalizing PureSVD for Effective and Efficient Top-N Recommendations - [<http://arxiv.org/abs/1510.02786v4>] Recovering a Hidden Community Beyond the Kesten-Stigum Threshold in $O(|E| \log^* |V|)$ Time - [<http://arxiv.org/abs/1503.04335v2>] An algebraic approach to finite projective planes - [<http://arxiv.org/abs/1711.02729v2>] On f- and h- vectors of relative simplicial complexes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=-9, elapsed=103.8s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_xor_and_network(n):
        network = []
        for _ in range(2**n - 1):
            node = [random.choice([0, 1]) for _ in range(n)]
            network.append(node)
        return network

    def tensor_product_algebra(network):
        n = len(network[0])
        algebra = [[0] * (2 ** n) for _ in range(2 ** n)]
        for i in range(2 ** n):
            for j in range(2 ** n):
```

```

        product = 1
        for k in range(n):
            if network[i][k] == network[j][k]:
                product *= 1
            else:
                product *= -1
        algebra[i][j] = product
    return algebra

def minimal_rank(matrix):
    m, n = len(matrix), len(matrix[0])
    rank = 0
    for i in range(m):
        if all(matrix[j][i] == 0 for j in range(i, m)):
            continue
        pivot_row = next(j for j in range(i, m) if matrix[j][i] != 0)
        matrix[i], matrix[pivot_row] = matrix[pivot_row], matrix[i]
        rank += 1
        for j in range(m):
            if i == j:
                continue
            factor = matrix[j][i] / matrix[i][i]
            for k in range(n):
                matrix[j][k] -= factor * matrix[i][k]
    return rank

def communication_complexity(network):
    n = len(network[0])
    # Placeholder for actual communication complexity calculation
    return (n ** 2) * math.log(n, 2) / math.log(math.log(n, 2), 2)

n = random.choice([5, 10, 15, 20, 30, 40])
network = generate_xor_and_network(n)
algebra = tensor_product_algebra(network)
rank = minimal_rank(algebra)
comm_complexity = communication_complexity(network)

return {
    "metric_name": "communication complexity",
    "metric_value": comm_complexity,
    "instances_tested": 1,
    "conjecture_holds": rank <= 10, # Placeholder for actual constant c
    "counterexample": "" if rank <= 10 else f"rank too high: {rank}"
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_comm_complexity = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_comm_complexity} std=0.0 support_fraction={support_fr
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"rank too high\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that neither the support nor falsification conditions could be clearly evaluated. | next: Re-run the test to ensure it completes successfully and provides the necessary data for evaluating the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14699
2	preregistration	ollama_remote	glm4:latest	0	0	12958
3	novelty	ollama_remote	glm4:latest	0	0	8456
4	novelty	ollama_remote	glm4:latest	0	0	9941
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13655
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10280
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8472
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9531
9	judge	ollama_remote	glm4:latest	0	0	72905

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 160897 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/a62ecea5349a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a62ecea5349a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a62ecea5349a.tar.gz` (if generated)